Research Loops and MCP, the execution model

Session 3. 40 minutes. One research assistant, built end to end.

Not a survey of nine frameworks.

Same graph. New object.

- Orchestrator, doer, judge. The exact three parts from Module 1.
- The object is a question. The artifact is a cited brief.
- The judge still holds no write path.
- The only new thing is a tool that reaches outside the machine.

A safe tool boundary is narrow, and it is read-only.

Model Context Protocol (MCP) is how the agent reaches outside itself.

- **Allowed**: search, and write into this loop's own output folder.
- **Denied**: merge, deploy, ticket state, anything in production.

A narrow schema beats a broad one. `add_review_comment(issue_id, body)` is a tool. An HTTP client holding your credentials is a liability.

Agents reach for the bigger tool. This is measured.

ToolPrivBench, 2026:

- Mainstream agents **often chose a higher-privilege tool** when a lower one was enough.
- Transient failures made that escalation **more likely**, not less.
- Prompt-based controls gave **only limited** mitigation.

You do not fix this with a stronger sentence.
You fix it by not shipping the

What comes back from a tool is untrusted input.

AgentDojo showed that content returned by a tool can carry instructions, and that those instructions can redirect the agent.

Your search results are a **document the internet wrote**, not a system prompt.

> Authorization is a property of the tool boundary, not a sentence in the system prompt.

The MCP authorization spec makes the same call: validate the token audience server side, and never pass a token through.

One boundary. Three backends. You pick.

Backend	When
Perplexity over MCP	You set `PERPLEXITY_API_KEY`
Your agent's own WebSearch	No key, but the tool is there
A recorded fixture	Offline, or the wifi in this room

The loop calls one function. It never learns which one answered.

Lab 3. The research assistant. 25 minutes.

```
cd labs/m3-research
claude -p "$(cat prompts/claude-code.md)"     # or codex, grok, opencode

task loop:research -- --question "sqlalchemy nullable datetime column" \
  --backend fixture
```

Fill `loop.py`. Two functions: `plan_questions` and `check_brief`.

The question is boring on purpose. This is not "write my next post."

Falling behind is fine: `git checkout done-m3`.

The judge reads the brief. It does not read it *thoughtfully*.

```
PASS  has_sources    2 sources retrieved
PASS  grounded       every citation resolves
PASS  cited          every paragraph cites a source
PASS  style          0 em dashes

backend: fixture   budget: $0.00 / $0.20 (soft $0.10), 3/8 calls
gate:    pass
```

Grounded and cited are **arithmetic**. No model call.

A confident sentence nobody can trace is the failure that matters.

A research loop needs a harder stop than code does.

"Keep searching until confident" is not a stop condition. The search space has no end, so the loop has to be told where the end is.

- **Call budget.** Eight searches. The ninth raises.
- **Dollar budget.** A soft warning, then a hard cap.
- **Stable failure.** The same gaps twice means stop.
- **No source found** escalates. It never ships

Two numbers worth remembering.

Number	What it says
15.7%	Share of recorded agent failures that are one step, repeated
12.4%	Share where the agent did not know it was already done

And retries are not linear. A retry usually replays the whole context, so a 20% per-step failure rate can roughly **double** the bill, not add a fifth to it.

Cost is an architecture problem, not a pricing problem.

Break. 15 minutes.

Next: the same stack, with nobody at the keyboard.